consumer version. Currently, we've set the `mainBranch` and `matchingBranch` properties to `true`. Another property that we can set is the `deployedOrReleased` property. To do that, let's update the provider/provider-contract-spec.js file:

```
const options = {
  …
  consumerVersionSelectors: [
    { mainBranch: true },
    { matchingBranch: true },
    { deployedOrReleased: true }
  ],
  …
};
```

Once the provider verification runs, it will verify contracts from the consumer from the main branch, contracts from any matching branch names, and contracts that have been deployed or released to an environment. When code changes are being merged by multiple developers and deployed to production, how can we ask Pact to ensure that the versions we're about to deploy to production are compatible with the ones already in production? You can look at the broker manually and check if the versions are compatible, but this could be time-consuming, especially as your contract testing efforts scale up. To check this automatically, let's use the cool Pact feature called can-i-deploy that we've mentioned a few times earlier in this book.

10.2.5 Using the can-i-deploy tool

The can-i-deploy tool is a feature introduced by Pact that lets you query the matrix table to verify if certain versions ready to be deployed to an environment have a successful verification result with the version already deployed in the same environment. This is useful when your team wants that extra safety net to ensure they can confidently deploy their changes to production. In addition, this can also help the team move to a CD model where a code is automatically deployed to production when all the jobs pass.

You might work in an organization where teams deploy different changes to a main branch, but the changes aren't frequently deployed to production. In such cases, there could be scenarios where the code deployed to